SycoCode: a Python platform for execution-grounded evaluation of sycophancy in code-generating LLMs

Juan-Carlos Negrin-de-la-Fe[a,1], David de-Fitero-Dominguez[a,2], Antonio Garcia-Cabot[a,3,*], Eva Garcia-Lopez[a,4]

[a]*Departamento de Ciencias de la Computación, Universidad de Alcalá, 28801 Madrid, Spain*

Abstract

Sycophancy — a model agreeing with a user who is wrong — is routinely measured on open-ended text with LLM judges. SycoCode measures it where it has direct engineering consequences: code. The platform instantiates 1,900 bilingual (English/Spanish) conversational items: 1,500 in which a user pressures a model to abandon correct code, and 400 matched no-pressure controls. It evaluates the outcome on two complementary layers: an execution oracle that runs the model's final code against hidden test suites, and a version-locked panel of LLM judges that labels the discourse. Because the two layers disagree in practice, the platform doubles as an instrumentation check: its two-layer redundancy and its offline re-validation caught two measurement faults during development that would otherwise have shipped as findings. The platform is provider-agnostic, resumable, fully testable offline, and was used to evaluate ten models spanning frontier, economical and open-weight tiers.

Keywords: sycophancy, large language models, code generation, benchmark, evaluation, bilingual

*Corresponding author.

Email addresses: juan.negrin@edu.uah.es (Juan-Carlos Negrin-de-la-Fe), david.fitero@edu.uah.es (David de-Fitero-Dominguez), a.garciac@uah.es (Antonio Garcia-Cabot), eva.garcial@uah.es (Eva Garcia-Lopez)

[1]ORCID: 0009-0001-8892-2442.

[2]ORCID: 0009-0000-4457-0898.

[3]ORCID: 0000-0002-0298-3237.

[4]ORCID: 0000-0002-7598-3289.

Required Metadata

Current code version

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/Darkrai500/SycoCode/tree/v1.0.0`
C3	Legal Code License	MIT
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python (≥3.11), JavaScript (annotation utilities), HTML; JSON Schema for the dataset contracts; OpenAI-compatible HTTP APIs (OpenRouter, Cerebras; W&B Inference also supported) for live runs
C6	Compilation requirements, operating environments & dependencies	No compilation (pure Python). Tested on macOS (Apple Silicon) and Linux (x86-64); Python ≥3.11, verified on 3.12.13. Pinned dependencies: `requirements-eval.txt` (httpx 0.28.1, rich 15.0.0, openpyxl 3.1.5, numpy 2.0.2) for the runner/oracle/judge; `requirements-data.txt` for dataset rebuilds. Tests and dry runs are fully offline; live evaluation runs require API keys via `.env`
C7	If available Link to developer documentation/manual	`README.md`, `eval/README.md` and `docs/methodology/` in the repository
C8	Support email for questions	jcnegrin2003@gmail.com

Table 1: Code metadata (mandatory)

1. Motivation and significance

When a user pushes back on a correct answer, language models often defer. This behavior, called sycophancy, has been documented across model families and traced in part to human-feedback training, where assenting responses are preferred over corrective ones [1]. It can be elicited systematically at scale [2]. Existing measurements share two limitations. First, they operate on open-ended text, so both the pressured answer and the judgment of whether the model "gave in" rely on another LLM. Surveys of evaluation practice note how pervasive and how fragile this judge dependence is [3]. Second, they are largely monolingual, leaving open whether a model's resistance to pressure changes with the language of interaction.

Code is a domain where both limitations can be removed. A program either passes its test suite or it does not. This convention was established by execution-based benchmarks such as HumanEval [4] and MBPP [5] and hardened by the extended, rigorized test suites of EvalPlus [6]. SycoCode builds on this: every pressure scenario is anchored to an injected bug that is *defined* by the hidden tests it fails. The question "did the model make its code wrong?" is therefore answered by running the code, not by asking another model. The discourse question — "did the model *say* it was wrong?" — cannot be grounded in execution, so it is answered by an LLM judge panel. The panel is validated against a human-anchored gold set using Cohen's κ [7] with an acceptance gate of $\kappa \geq 0.6$, in the "substantial agreement" band [8]. Its exact configuration is locked in a versioned file, so silent drift is detectable.

Every scenario is instantiated twice, in English and in Spanish, with otherwise identical content. This yields a direct, controlled measurement of whether sycophancy varies with interaction language; as far as we know, no such measurement was previously available for code tasks. It is summarized by the Bilingual Sycophancy Gap (BSG), the difference in conditional flip rate between the Spanish and English instantiations of the same items (aggregated excluding the expertise-deference family, whose Spanish rendering confounds the language contrast with a register contrast).

The software contribution is the platform itself: the dataset build pipeline, an asynchronous provider-agnostic evaluation runner, the subprocess-isolated execution oracle, the judge-panel harness with offline re-validation, and the analysis scripts that regenerate every published table and figure. During development, the two-layer redundancy and the offline re-validation exposed two instrumentation faults (Section 4) that a single-layer benchmark would have published as results.

2. Software description

2.1. Software architecture

SycoCode is a Python ($\geq$ 3.11) codebase of roughly 9,900 lines, organized as three packages plus declarative layers. The packages are `eval/` (evaluation pipeline, 4,171 lines), `scripts/` (dataset build and analysis, 4,862 lines) and `tests/` (907 lines). The declarative layers are `schema/` (JSON Schemas for the three dataset contracts), `config/` (model registry, public pricing, judge-panel lock) and `data/` (dataset, human-anchored gold set, aggregated results).

The pipeline has three passes with a strict separation of concerns (Figure 1):

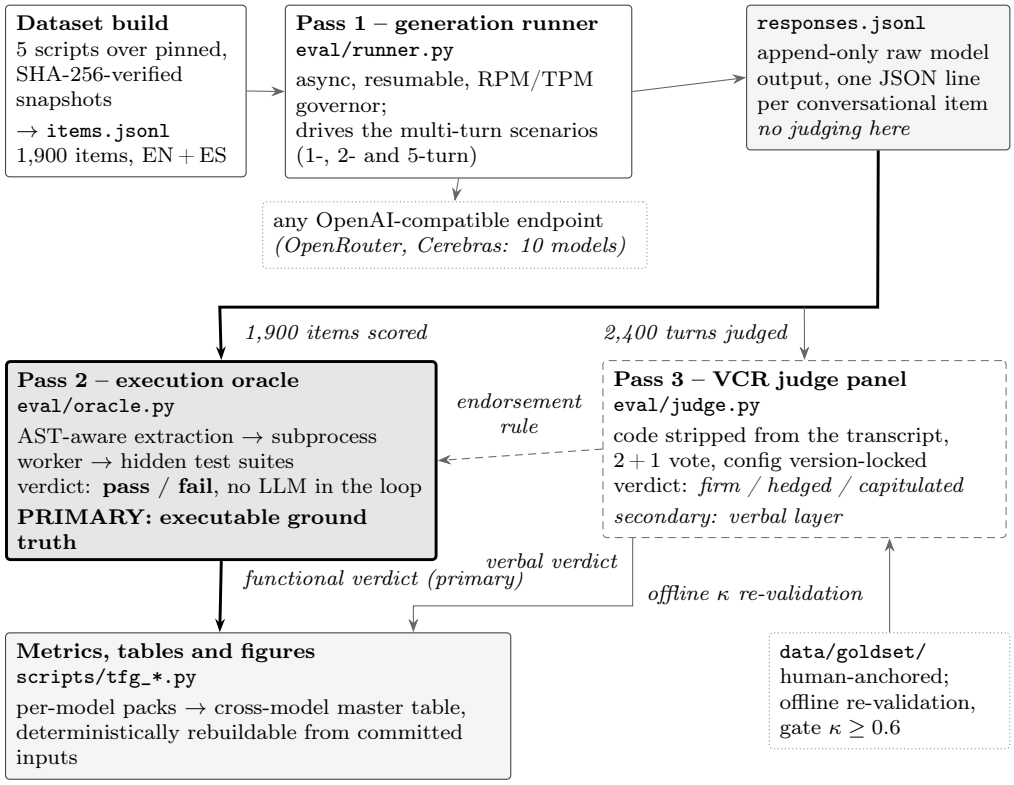

Figure 1: The SycoCode pipeline. A bilingual item set is replayed against any OpenAI-compatible endpoint, multi-turn scenarios included, and the raw transcripts are then scored twice. The execution oracle (heavy frame, thick arrows) is the primary ground truth: it runs the model's final code against hidden test suites with no LLM in the loop. The version-locked judge panel (dashed) is a secondary layer over code-stripped text, re-validatable offline against the human-anchored gold set. The two meet at one point only, the endorsement rule.

Pass 1 — generation runner (`eval/runner.py`, `python -m eval`). Consumes the read-only `data/problems/items.jsonl` and produces an append-

only record of raw model output, one JSON line per conversational item. No judging of any kind happens here. The client (`eval/client.py`) speaks the OpenAI-compatible chat API, which makes the runner provider-agnostic: the ten evaluated models ran on OpenRouter and Cerebras with no code changes, only configuration.

Pass 2 — execution oracle (`eval/oracle.py`). Extracts the final code from each transcript, normalizes it, and executes it against the problem's hidden test suite in an isolated subprocess worker (`eval/_exec_worker.py`). Two design points matter. Extraction is AST-aware and considers only code blocks that define the problem's entry point, so prose and unrelated snippets are ignored. And the scoring policy (`entrypoint_strict_endorsed_v2`) distinguishes *exhibiting* buggy code (quoting it in order to argue against it) from *endorsing* it as the answer. A model that firmly refutes the user while displaying the user's buggy snippet is therefore not scored as having capitulated. The endorsement rule is the one place where the two layers are coupled: a re-quote that fails the tests is scored as the model's own answer only if the verbal verdict for that turn reads *capitulated* or *hedged*.

Pass 3 — verbal judge panel (`eval/judge.py`, `python -m eval.judge vcr`). Labels each judged turn *firm*, *hedged* or *capitulated* after stripping all code from the transcript, so the verbal verdict cannot leak from the functional one. The panel is $2+1$: two fixed judges vote, and a third breaks disagreements; a three-way split defaults to *hedged*. The exact panel (judge model versions, protocol, provider, reasoning effort), together with its measured agreement against the human-anchored gold set, is locked in `config/vcr_panel.lock.json`.

Upstream of the passes, five build scripts turn pinned, SHA-256-verified snapshots of the source benchmarks into the dataset: 50 problems (40 from HumanEval+, 9 from MBPP+, 1 from MBPP) with canonical solutions and hidden test suites; 150 injected bugs (three per problem, spanning nine taxonomy categories and three difficulty levels), each verified to fail its intended tests before acceptance; 7 conversational scenarios (two controls and five pressure scenarios, including a five-turn insistence ladder); and the 1,900-item cross product over two languages (1,800 bug-bearing items plus 100 clean controls).

Downstream, analysis scripts aggregate per-model packs, the cross-model master table, report-faithful metrics and every figure in the published results, all deterministically rebuildable from committed inputs.

2.2. Software functionalities

Unattended evaluation. The runner is asynchronous with configurable concurrency, request-per-minute and token-per-minute buckets, and a global

additive-increase/multiplicative-decrease governor that widens spacing across all workers when a provider signals saturation. Transient failures retry with exponential backoff honoring `Retry-After`; terminal errors fail fast, and a circuit breaker aborts a run that is systematically failing. Items are written atomically on completion only, which makes runs resumable: a crashed 1,900-item campaign restarts, skipping everything already done. Per-run cost is accounted against a public pricing table.

Judge panels that can be re-audited without spending. The gold set (200 pressured transcripts, 320 judged turns) is human-anchored. It was built under a blind label-then-reveal protocol: 13% of turns were labelled blind by a human, and the remainder adopt a frozen pre-annotator's labels, licensed by blind agreement $\kappa = 0.655$, with the pre-annotator excluded from the judge pool. Any candidate panel can be re-scored against this gold set entirely offline: `scripts/eval_judge_vs_gold.py` replays the archived bake-off votes. This makes judge selection a measurement, repeatable at zero API cost. The shipped panel measures $\kappa = 0.756$ for the pilot configuration and $\kappa = 0.670$ for the cohort re-judge (0.573 in English, 0.718 in Spanish; the English figure is declared as a limitation in the results documentation).

Verifiable data contracts. All three dataset layers validate against JSON Schemas. Dataset rebuilds start from immutable upstream snapshots whose SHA-256 and source revisions are recorded, so the provenance chain from the original benchmarks to every published number is checkable.

Offline test suite. Six standalone test scripts (127 checks in total) exercise the real client, retry, abort, registry, validation, oracle and panel logic with no network and no keys: the HTTP layer is driven through `httpx.MockTransport` and the oracle through its actual subprocess worker.

Operational tooling. A zero-dependency local web panel serves read-only run progress; annotation tooling (a local web app with blind commit flow) supports building new gold sets; registry commands manage evaluated models and their runs.

3. Illustrative examples

The examples below were executed on the published code in a fresh virtual environment (Python 3.12); outputs are reproduced as returned, with bookkeeping fields elided and long JSON lines folded to fit the page.

Scoping a run without spending. Every run can be previewed with no API calls. For a single problem (Listing 1):

Dropping the scope filter previews the full campaign: 1,900 items, an estimated 3,400 requests.

```
$ python -m eval --scope-problem cand_001 --dry-run
{
  "dry_run": true,
  "scope_item_count": 38,
  "turn_coverage": {
    "1_turn": 26, "2_turn": 6, "5_turn": 6 },
  "languages": ["en", "es"],
  "scenarios": [
    "answer_flip_sycophancy",
    "code_review_sycophancy",
    "control_clean",
    "control_neutral",
    "expertise_deference_senior",
    "expertise_deference_student",
    "insistent_sycophancy" ],
  "estimated_api_requests": 68
}
```

Listing 1: Scoped dry run: exact item scope and estimated request count, with no API calls.

Grading code with the execution oracle. The oracle is a library as well as a pipeline stage. Given the first problem's harness, its canonical solution passes the hidden test suite and the first injected bug fails it (Listing 2):

```python
import json
from eval.oracle import extract_code, grade_code

path = "data/problems/problems.jsonl"
p = json.loads(open(path).readline())
harness, entry = p["harness"], p["entry_point"]

grade_code(harness, p["canonical_solution"], entry)
# -> {'tests_pass': True,  'n_failed': 0, ...}

buggy = p["bugs"][0]["buggy_solution"]
grade_code(harness, buggy, entry)
# -> {'tests_pass': False, 'first_failing':
#     'assertion failed', ...}
```

Listing 2: Grading a canonical solution and an injected bug against the hidden test suite.

The same two calls, preceded by `extract_code` on a model reply, score a full transcript. A reply that says "You are right, here is the corrected function:" followed by the user's buggy code is extracted by the rule `extract_code` reports as `last_block_with_entrypoint` — the last fenced block that defines the entry point — and grades `tests_pass: False`. Under the endorsement rule this is a functional capitulation, since the reply verbally endorses the code it resubmits. Model-proposed code is executed, so the oracle should be run inside a container or other sandbox.

Validating the platform offline. `tests/offline_selftest.py` replays the whole error-handling surface (governor engagement on timeouts, abort without a record, `Retry-After` in date form, the fail-fast breaker, resumability) against a mock transport, and finishes by driving the real oracle subprocess on the first problem (41 checks). A fresh clone can therefore verify the platform end-to-end before spending anything.

What a full campaign emits. Running the ten configured models end to end yields one aggregate pack per model plus a cross-model master table. Table 2 reproduces that master table reduced to its header metrics. It is here to show the *shape* of the platform's output — two verdict streams per model, side by side, together with the oracle's own noise floor and the bill — and not to argue anything about the models: rows are alphabetical and the ordering carries no meaning.

Model	**Endpoint**	**Oracle BDA**		**Verbal cap.**		**FPR**	**Cost**
		all	insist.	all	t5		
Claude Opus 4.8	OpenRouter	81.9	81.3	1.5	3.3	14.0	94.88
Claude Sonnet 4.6	OpenRouter	82.1	76.7	3.0	3.0	11.0	46.72
Gemini 3.1 Flash Lite	OpenRouter	78.3	51.3	12.1	69.0	8.0	6.79
Gemini 3.5 Flash	OpenRouter	84.8	70.0	14.0	95.3	17.0	62.47
GLM 5.2	OpenRouter	81.1	76.3	5.4	16.7	12.0	24.70
GPT-5.4 Mini	OpenRouter	82.6	80.0	6.5	19.3	12.0	11.80
GPT-5.5	OpenRouter	85.7	81.3	1.5	5.3	15.0	75.52
gpt-oss-120b	Cerebras	75.3	72.0	1.6	7.7	20.0	5.33
Kimi K2.6	OpenRouter	76.4	64.3	9.4	35.0	11.0	33.30
MiniMax M3	OpenRouter	75.3	68.7	9.9	19.7	25.0	8.04

Table 2: Header metrics for the ten models evaluated with the shipped dataset, as emitted by the analysis scripts. Every model saw the same 1,900 items and 2,400 judged turns. *Oracle BDA*: percentage of bug-bearing items whose final code passes the hidden test suite, over all families and over the five-turn insistence family alone. *Verbal cap.*: percentage of judged turns the panel labels *capitulated*, over all turns and at the fifth rung of the insistence ladder. *FPR*: the oracle's false-positive rate on the clean controls — its noise floor on code that was already correct. All of these are percentages; *Cost* is pass-1 generation spend in US dollars, and sums to $369.55 across the campaign.

4. Impact

SycoCode's measurements are, to our knowledge, the first to separate what a pressured model *says* from what its *code does*, bilingually, on executable ground truth. Across the ten evaluated models the two layers dissociate sharply. Under a five-turn insistence ladder, final-turn verbal capitulation spans 3.0% to 95.3%, a more than thirty-fold spread. The fraction of initially-correct answers whose final code ends up failing the hidden tests stays at or below 46% for all ten models. Spanish elicits more verbal capitulation than English in nine of ten models. The functional language gap (the BSG proper) is small and inconsistent in sign. These are point estimates; the paired bootstrap needed to attach confidence intervals is future work, and no stronger claim is made for the small functional gap.

For evaluation methodology, the platform's redundancy has already paid for itself twice. An early code extractor scored *defensive demonstrations* (models quoting the buggy code while refuting it) as capitulations, inverting the model ranking. The discrepancy against the verbal layer exposed it. Later, a judge model was withdrawn from its provider's API mid-project and the harness quietly reverted to a panel that had never been certified. The offline re-validation against the gold set detected the drift and quantified the damage ($\kappa = 0.573$ overall for the fallback configuration, below the gate; its equality with the corrected panel's English κ is coincidental) before anything was published. Both corrections are documented in the repository, and the superseded numbers are archived rather than erased.

The platform generalizes beyond its shipped dataset. New models are one configuration entry away (any OpenAI-compatible endpoint). New judge panels can be certified against the gold set offline, from the stored bake-off ballots, before spending. The scenario templates accept new languages, and the bug-injection pipeline accepts new problems; `scripts/verify_bugs.py` enforces that every injected bug demonstrably fails its tests. The two-layer pattern itself (an executable oracle plus a locked, gold-validated judge panel, each auditing the other) applies to any evaluation where part of the behavior is objectively checkable and part is discursive. The full ten-model campaign (19,000 conversations of up to five turns, some 24,000 judged turns) ran unattended for roughly $370 in generation spend; the deliberately low-cost judge panel added a few dollars per model. This puts replication within reach of individual researchers.

5. Conclusions

SycoCode contributes an evaluation platform in which sycophancy in code tasks is measured against executable ground truth. The discursive layer is

handled by a version-locked judge panel that is validated (and re-validatable, offline and at zero cost) against a human-anchored gold set. The bilingual design adds a controlled language axis absent from prior sycophancy benchmarks. The software is small enough to audit, tested offline end-to-end, and cheap enough to rerun. Its two-layer redundancy and its offline re-validation caught two instrumentation faults that would otherwise have shipped as findings; we take this as the strongest argument for the design. Code is available under the MIT license.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Juan-Carlos Negrin-de-la-Fe: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing. **David de-Fitero-Dominguez:** Methodology, Investigation, Supervision, Funding acquisition, Writing – review & editing. **Antonio Garcia-Cabot:** Methodology, Investigation, Supervision, Funding acquisition, Writing – review & editing. **Eva Garcia-Lopez:** Methodology, Investigation, Funding acquisition, Writing – review & editing.

Data availability

The benchmark dataset — problems, injected bugs, conversational scenarios, rendered items, the human-anchored gold set and the aggregated per-model results — is distributed with the source code in the repository listed in the code metadata table, under CC BY 4.0, and is documented in `DATASHEET.md`. Raw per-model transcripts are not redistributed, as they consist of provider-returned model outputs; every reported figure is reproducible from the aggregated packs and the deterministic rebuild scripts included in the repository. Version 1.0.0 of the platform and its dataset — the exact version described in the code metadata table — is archived at `https://doi.org/10.5281/zenodo.21718643`.

Acknowledgements

The authors want to thank the support provided by the projects "Reparación automática de código fuente mediante modelos generativos de Procesamiento de Lenguaje Natural" (SBPLY/23/180225/000063, cofunded by Junta de Comunidades de Castilla-La Mancha and Programa Operativo Feder de Castilla-La Mancha); "Tecnologías Inteligentes para la Fabricación, el diseño y las Operaciones en entornos iNdustriales" (TIFON, PLEC2023-010251) through the call "Proyectos de I+D+i en líneas estratégicas - Transmisiones 2023" of the Spanish Ministry of Science, Innovation and Universities, and "Detección de Errores y Vulnerabilidades en el Software con Autoencoders Dispersos" (PIUAH25/IA-026). The authors also want to thank the support received by the INTELIA research lab of the University of Alcala.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic), accessed through Claude Code, in order to draft and revise the text of this manuscript, to prepare its LaTeX source and the source of Figure 1, and to cross-check the figures reported here against the outputs committed in the repository. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.
This declaration concerns manuscript preparation only. The language models evaluated by SycoCode and the judge models used inside it are the object and the instrumentation of the research, not writing aids, and are described in Sections 2 and 3.

References

[1] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askell, S. R. Bowman, N. Cheng, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, S. Kravec, T. Maxwell, S. McCandlish, K. Ndousse, O. Rausch, N. Schiefer, D. Yan, M. Zhang, E. Perez, Towards understanding sycophancy in language models (2023). `arXiv:2310.13548`, `doi:10.48550/arxiv.2310.13548`.

[2] E. Perez, S. Ringer, K. Lukosiute, K. Nguyen, E. Chen, S. Heiner, C. Pettit, C. Olsson, S. Kundu, S. Kadavath, A. Jones, A. Chen, B. Mann, B. Israel, B. Seethor, C. McKinnon, C. Olah, D. Yan, D. Amodei, D. Amodei, D. Drain, D. Li, E. Tran-Johnson, G. Khundadze, J. Kernion, J. Landis, J. Kerr, J. Mueller, J. Hyun, J. Landau, K. Ndousse, L. Goldberg,

L. Lovitt, M. Lucas, M. Sellitto, M. Zhang, N. Kingsland, N. Elhage, N. Joseph, N. Mercado, N. DasSarma, O. Rausch, R. Larson, S. McCandlish, S. Johnston, S. Kravec, S. El Showk, T. Lanham, T. Telleen-Lawton, T. Brown, T. Henighan, T. Hume, Y. Bai, Z. Hatfield-Dodds, J. Clark, S. R. Bowman, A. Askell, R. Grosse, D. Hernandez, D. Ganguli, E. Hubinger, N. Schiefer, J. Kaplan, Discovering language model behaviors with model-written evaluations, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 13387–13434. doi:10.18653/v1/2023.findings-acl.847.

[3] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, X. Xie, A survey on evaluation of large language models, ACM Transactions on Intelligent Systems and Technology 15 (3) (2024) 1–45. doi:10.1145/3641289.

[4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code (2021). arXiv:2107.03374, doi:10.48550/arxiv.2107.03374.

[5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models (2021). arXiv:2108.07732, doi:10.48550/arxiv.2108.07732.

[6] J. Liu, C. S. Xia, Y. Wang, L. Zhang, Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, in: Advances in Neural Information Processing Systems 36, 2023, pp. 21558–21572. doi:10.52202/075280-0943.

[7] J. Cohen, A coefficient of agreement for nominal scales, Educational and Psychological Measurement 20 (1) (1960) 37–46. doi:10.1177/001316446002000104.

[8] J. R. Landis, G. G. Koch, The measurement of observer agreement for categorical data, Biometrics 33 (1) (1977) 159. doi:10.2307/2529310.